

Part 3 死锁概念

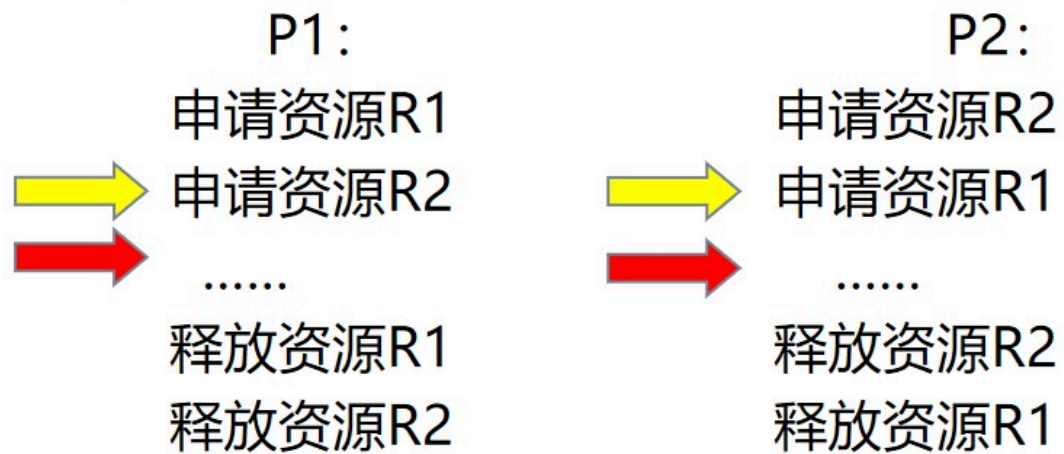
3.1 死锁概念

死锁 (Deadlock)：指多个进程在运行过程中因争夺资源而造成的一种僵局，当进程处于这种僵持状态时，若无外力作用，这些进程都将永远不能再向前推进。

Part 3 死锁概念

3.2 死锁产生的原因

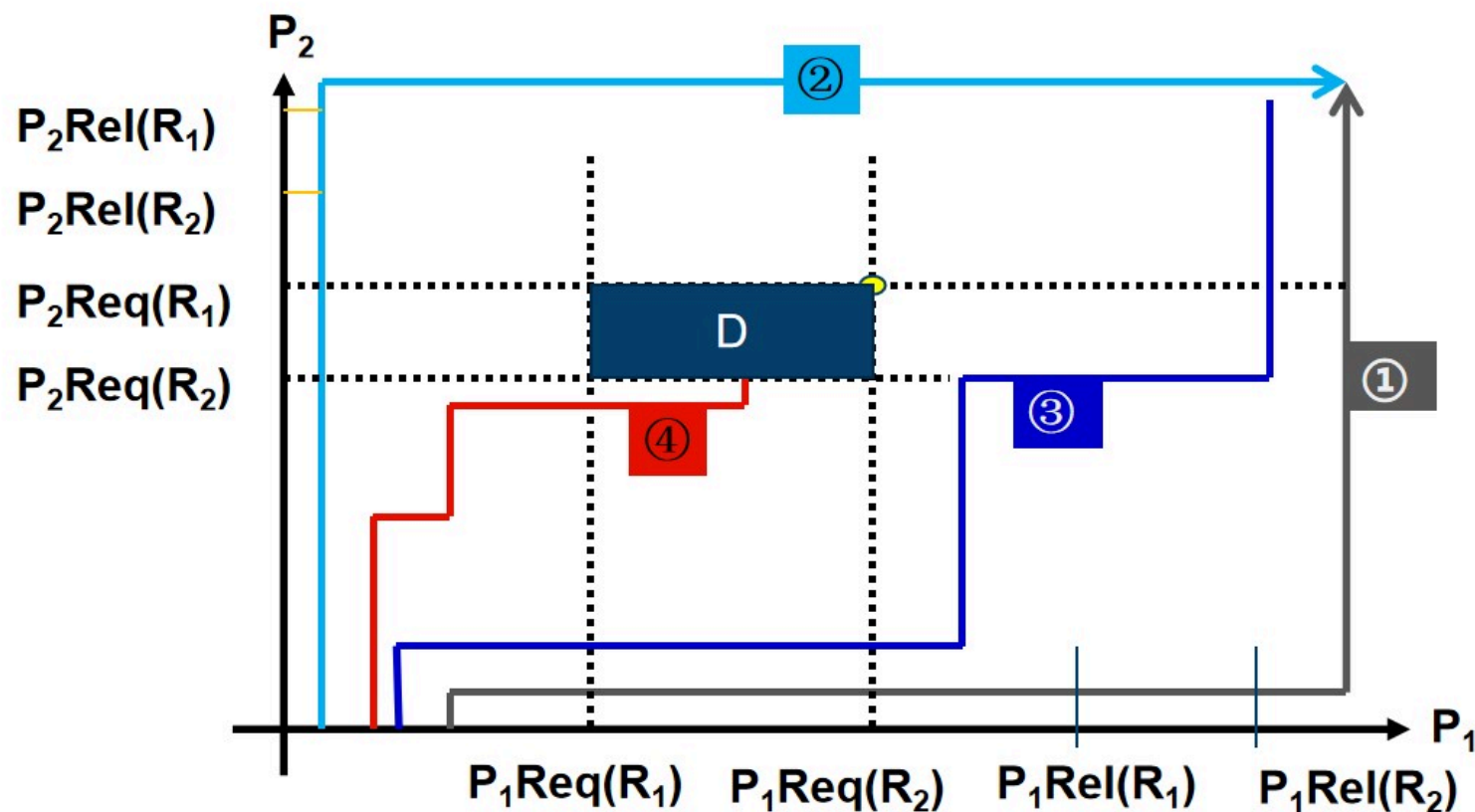
- 例3-1



Part 3 死锁概念

3.2 死锁产生的原因

进程推进顺序问题



Part 3 死锁概念

3.3 死锁的必要条件

- 互斥条件
- 请求和保持条件
- 不可抢占条件（不可剥夺）
- 循环等待条件（环路等待）

Part 3 死锁概念

3.3 死锁的必要条件

- **判断是否死锁的方法 (补充)**

系统中有N个进程，某种资源M个，每个进程需要最多k个资源，
系统不会发生死锁的条件是：

$$N(k-1) < M$$

Part 4 死锁处理方法



学习目标

- 能够理解死锁处理的几类方法
- 能够解释死锁预防的方法及局限性
- 能够判区分系统安全状态和不安全状态
- 能够应用银行家算法分析系统进行资源分配
- 能够应用资源分配图检测死锁

Part 4 死锁处理方法

处理死锁的方法

- 🏠 确保系统永远不会进入死锁状态
 - 死锁预防
 - 死锁避免
- 📁 允许系统进入死锁状态，然后恢复系统
 - 死锁检测 ➤ 死锁恢复
- 🔒 忽略这个问题，假装系统中从未出现过死锁。这个方法被大部分的操作系统采用，包括UNIX

Part 4 死锁处理方法

4.1 死锁的预防

- 破坏四个死锁的四个必要条件之一：

破坏“~~互斥~~”条件

破坏“请求和保持”条件

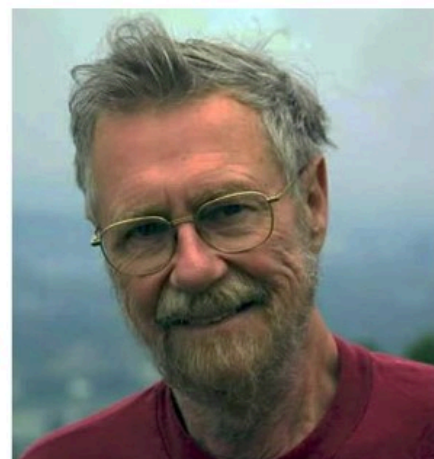
破坏“不可抢占”条件

破坏“循环等待”条件

Part 4 死锁处理方法

4.2 死锁的避免

Dijkstra



- 银行家拥有一笔周转资金
- 客户要求分期贷款，如果客户能够得到各期贷款，就一定能够归还贷款，否则就一定不能归还贷款
- 银行家应谨慎的贷款，防止出现坏帐

Part 4 死锁处理方法

4.2 死锁的避免

4个客户每个都有一个贷款额度

名字	已使用	最大
Andy	0	6
Barbara	0	5
Marvin	0	4
Suzanne	0	7

可用: 10

(a)

名字	已使用	最大
Andy	1	6
Barbara	1	5
Marvin	2	4
Suzanne	4	7

可用: 2

(b)

安全状态

名字	已使用	最大
Andy	1	6
Barbara	2	5
Marvin	2	4
Suzanne	4	7

可用: 1

(c)

不安全状态

Part 4 死锁处理方法

4.2 死锁的避免

- 用银行家算法避免死锁
 - 操作系统（银行家）
 - 操作系统管理的资源（周转资金）
 - 进程（要求贷款的客户）

Part 4 死锁处理方法

4.2 死锁的避免

- **安全状态和不安全状态**

- **安全状态：**是指系统能按某种进程顺序($P_1, P_2, \dots, P_n$)(称 $\langle P_1, P_2, \dots, P_n \rangle$ 序列为安全序列)，来为每个进程 P_i 分配其所需资源，直至满足每个进程对资源的最大需求，使每个进程都可顺利地完
- **不安全状态：**如果系统无法找到这样一个安全序列，则称系统处于不安全状态。

Part 4 死锁处理方法

4.2 死锁的避免

- 安全状态和不安全状态例

- 假定系统中三个进程P1、P2和P3，共有12台磁带机。进程P1总共要求10台磁带机，P2和P3分别要求4台和9台。假设在T0时刻：

进 程	最 大 需 求	已 分 配	可 用
P₁	10	5	3
P₂	4	2	
P₃	9	2	

Part 4 死锁处理方法

4.2 死锁的避免

- 多种资源的安全和不安全状态

进程	磁带机	绘图仪	打印机	CD-ROM
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

已分配的资源

进程	磁带机	绘图仪	打印机	CD-ROM
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

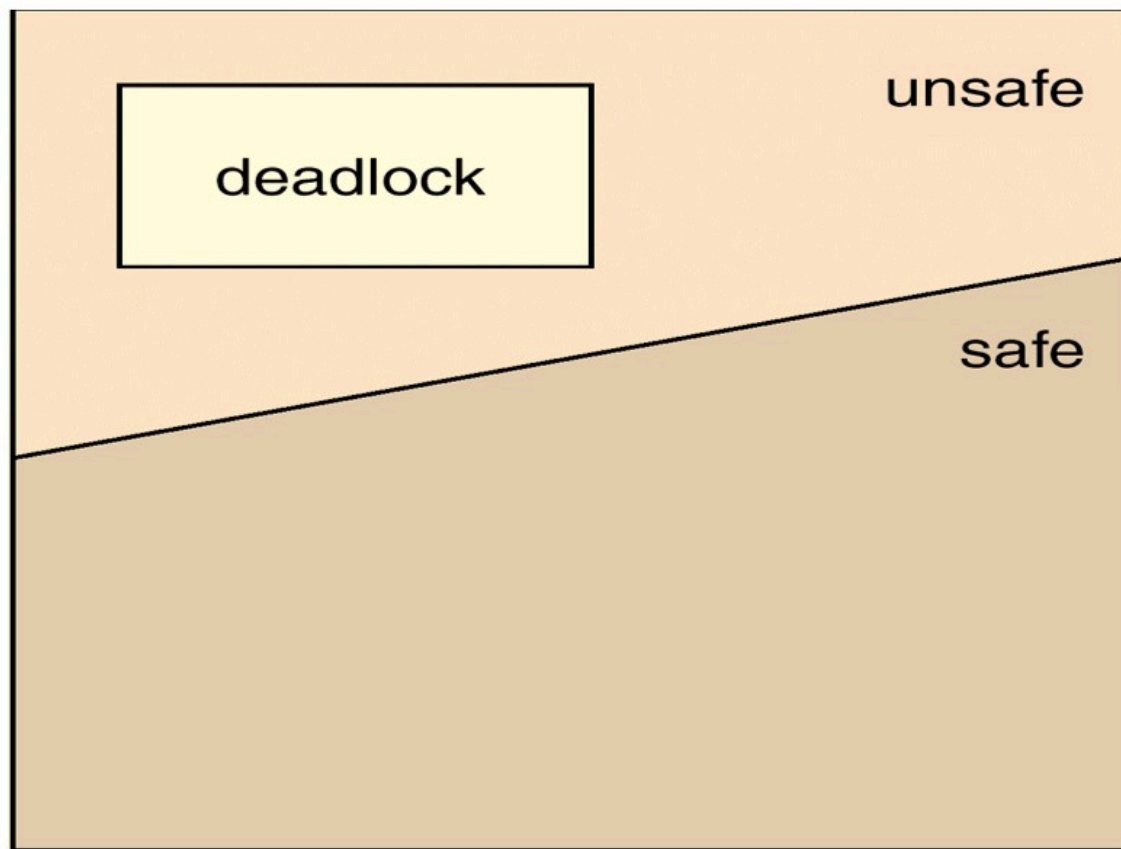
仍需要的资源

$E = (6342)$
 $P = (5322)$
 $A = (1020)$

Part 4 死锁处理方法

4.2 死锁的避免

- 安全、不安全、死锁状态空间



Part 4 死锁处理方法

4.2 死锁的避免



如果一个系统在安全状态，就没有死锁



如果一个系统处于不安全状态，就有可能死锁



死锁避免 $\Rightarrow$ 确保系统永远不会进入不安全状态

Part 4 死锁处理方法

4.3 银行家算法



Part 4 死锁处理方法

4.3 银行家算法

银行家算法的数据结构

n为进程的数目, m为资源类型的数目

-  **Available:** 长度为 m 的向量。如果 $available[j]=k$, 那么资源 R_j 有 k 个实例有效
-  **Max:** $n \times m$ 矩阵。如果 $Max[i,j]=k$, 那么进程 P_i 可以最多请求资源 R_j 的 k 个实例
-  **Allocation:** $n \times m$ 矩阵。如果 $Allocation[i,j]=k$, 那么进程 P_j 当前分配了 k 个资源 R_j 的实例
-  **Need:** $n \times m$ 矩阵。如果 $Need[i,j]=k$, 那么进程 P_j 还需要 k 个资源 R_j 的实例

$$Need[i,j] = Max[i,j] - Allocation[i,j].$$

Part 4 死锁处理方法

4.3 银行家算法

- ◆ 5个进程 P_0 到 P_4 , 3个资源类型: A(10个实例), B (5个实例), C (7个实例)
- ◆ 时刻 T_0 的快照:

	<u>Allocation</u>			<u>Max</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2
P ₁	2	0	0	3	2	2	Need		
P ₂	3	0	2	9	0	2	A	B	C
P ₃	2	1	1	2	2	2	7	4	3
P ₄	0	0	2	4	3	3	1	2	2
							6	0	0
							0	1	1
							4	3	1

Part 4 死锁处理方法

4.3 银行家算法

- 系统处在安全的状态，因为序列 $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ 满足了安全的标准
- Available=3 3 2, Work=Available

资源情况 进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₁	3	3	2	1	2	2	2	0	0	5	3	2	true
P ₃	5	3	2	0	1	1	2	1	1	7	4	3	true
P ₄	7	4	3	4	3	1	0	0	2	7	4	5	true
P ₂	7	4	5	6	0	0	3	0	2	10	4	7	true
P ₀	10	4	7	7	4	3	0	1	0	10	5	7	true

Part 4 死锁处理方法

4.3 银行家算法

安全性算法

```
boolean function safe(state:s; /*banker's algorithm*/
    int work[m]; //工作向量
    boolean finish [m]; //进程是否能够获得足够资源, 运行完成
    work=available;
    for (i=0;i<n;i++) finish[i]=false;    possible=true;
    while possible do
    { find a Pk such that
      { finish[k]=false; Need[k,j]≤work[j]; }
      if found then
      { work=work+Allocation[k];
        finish[k]=true; }
      else possible=false;
    }
    if all finish[i]==true safe=true;
    else safe=false;
```

Part 4 死锁处理方法

4.3 银行家算法

例子续: P1 request (1,0,2)

检查 $\text{Request} \leq \text{Available}$, 就是说, $(1,0,2) \leq (3,3,2)$ 为真

	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	4	3	2	3	0
P ₁	3	0	2	0	2	0			
P ₂	3	0	2	6	0	0			
P ₃	2	1	1	0	1	1			
P ₄	0	0	2	4	3	1			

Part 4 死锁处理方法

4.3 银行家算法

进程 \ 资源情况	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₁	2	3	0	0	2	0	3	0	2	5	3	2	true
P ₃	5	3	2	0	1	1	2	1	1	7	4	3	true
P ₄	7	4	3	4	3	1	0	0	2	7	4	5	true
P ₀	7	4	5	7	4	3	0	1	0	7	5	5	true
P ₂	7	5	5	6	0	0	3	0	2	10	5	7	true

执行安全算法表明序列 $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ 满足要求

补充思考:

- P₄的请求(3,3,0)是否可以通过? ➤ P₀的请求(0,2,0)是否可以通过?

Available

A B C

2 3 0

Part 4 死锁处理方法

4.3 银行家算法

银行家算法

```
if Requesti[j] > Need[i,j]    <error>
else
    if Requesti[j] > Available[j]    <suspend process.>
    else
        { define newstate by:
            Allocation[i,j] := Allocation[i,j] + Requesti[j]
            Available[j] := Available[j] - Requesti[j]
            Need[i,j] := Need[i,j] - Requesti[j]
        }
        if safe(newstate)        { carry out allocation }
    else
        { restore original state }
        { suspend process }
```

Part 4 死锁处理方法

3.3 死锁的必要条件

- **判断是否死锁的方法 (补充)**

系统中有N个进程，某种资源M个，每个进程需要最多k个资源，系统不会发生死锁的条件是：

$$N(k-1) < M$$

Part 4 死锁处理方法

4.4 死锁检测

资源分配图

系统模型

- 🏠 资源类型 $R_1, R_2, \dots, R_m$
 - CPU周期, 内存空间, I/O设备
- 📦 每一种资源 R_i 有 W_i 种实例
- 🔒 每一个进程通过如下方法来使用资源
 - 申请 ➤ 使用 ➤ 释放

Part 4 死锁处理方法

4.4 死锁检测

资源分配图

一个顶点的集合V和边的集合E



V被分为两个部分

- $P = \{P_1, P_2, \dots, P_n\}$, 含有系统中全部的进程
- $R = \{R_1, R_2, \dots, R_m\}$, 含有系统中全部的资源



请求边: 有向边 $P_i \rightarrow R_j$

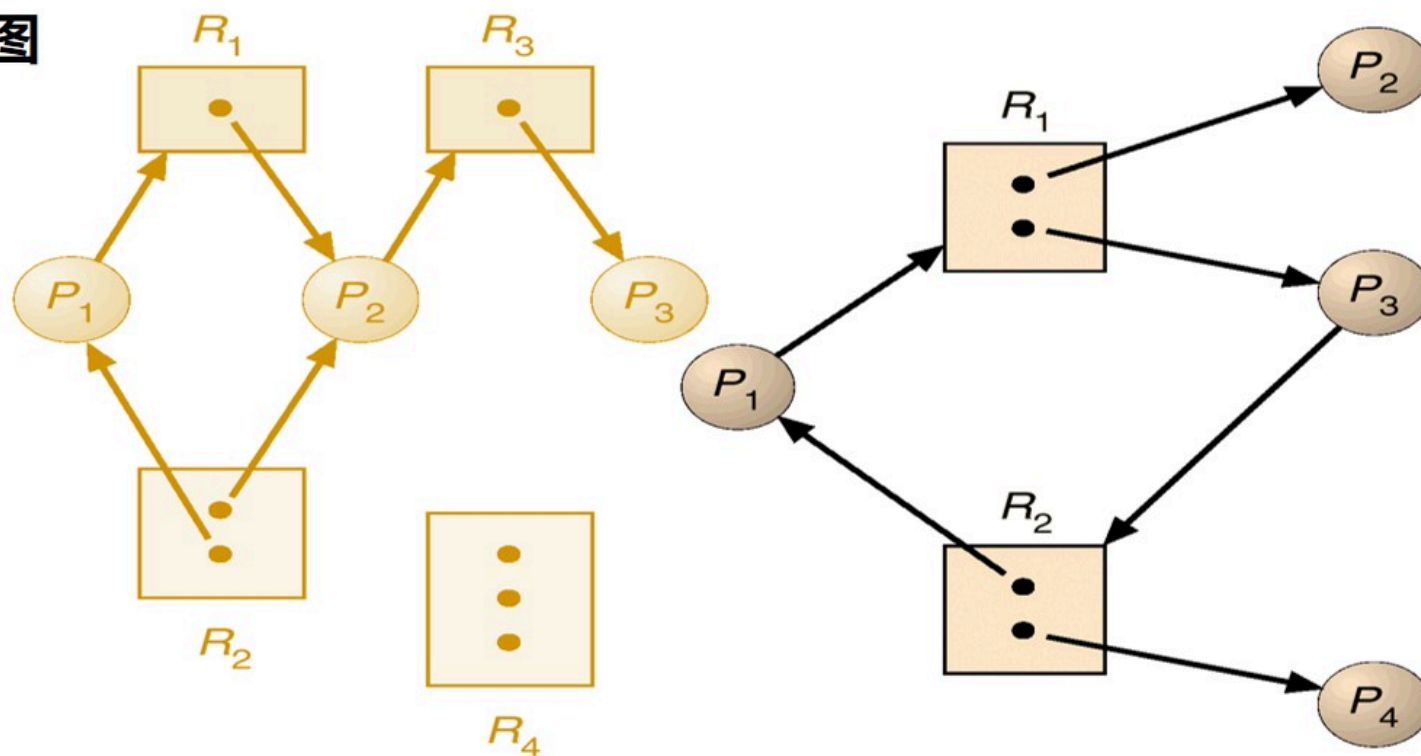


分配边: 有向边 $R_j \rightarrow P_i$

Part 4 死锁处理方法

4.4 死锁检测

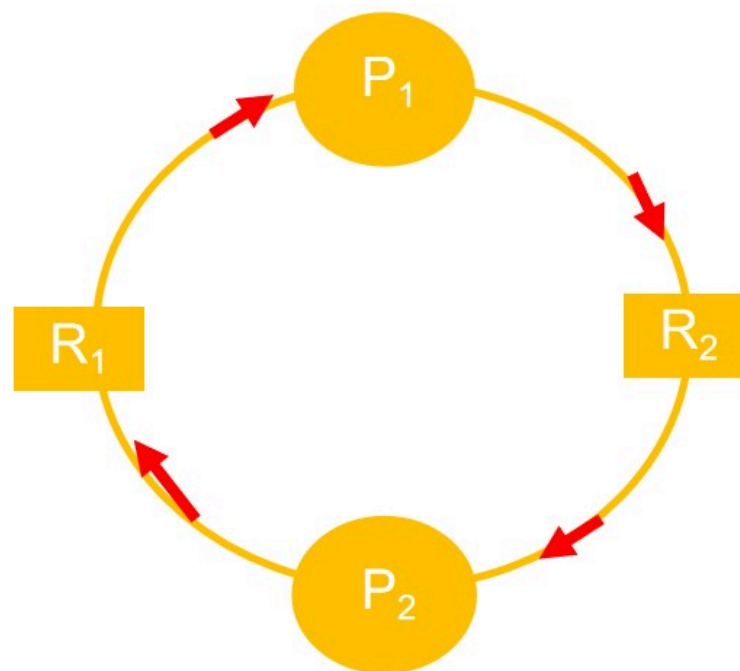
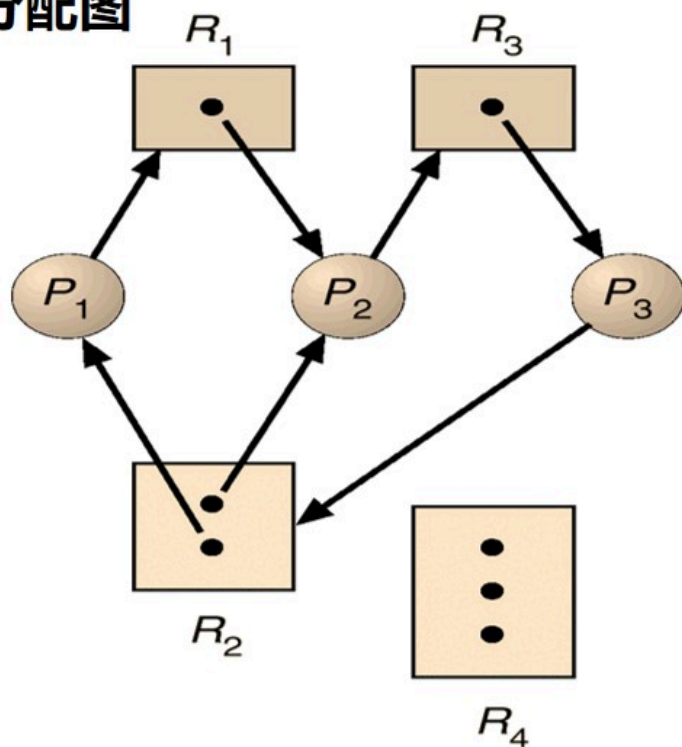
资源分配图



Part 4 死锁处理方法

4.4 死锁检测

资源分配图



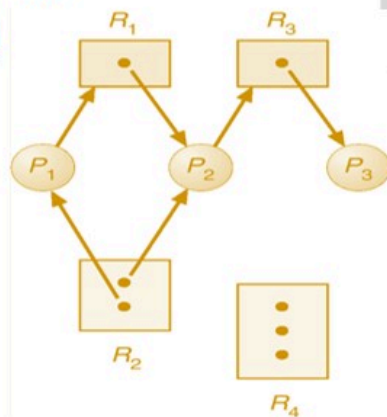
Part 4 死锁处理方法

4.4 死锁检测

资源分配图判断死锁

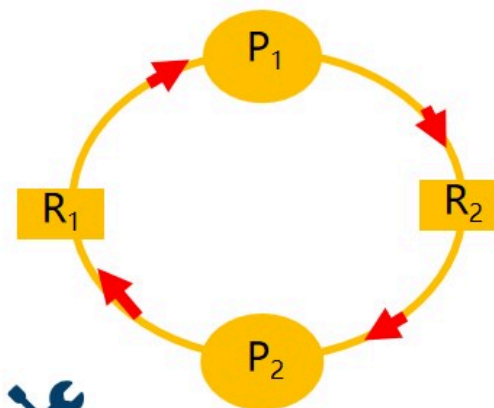


如果图没有环，那
么不会有死锁！



如果图有环，那么：

- 如果每一种资源类型只有一个实例，那么死锁发生；
- 如果一种资源类型有多个实例，那么可能死锁。



Part 4 死锁处理方法

4.4 死锁检测

资源分配图判断死锁

01

在资源分配图中，找出一个既不阻塞又非独立的进程结点 p_i 。在顺利的情况下， p_i 可获得所需资源而继续运行，直至运行完毕，再释放其所占有得全部资源，这相当于消去 p_i 所有的请求边和分配边，使之成为孤立的结点；

02

p_1 释放资源后，便可使 p_2 获得资源而继续运行，直到 p_2 完成后又释放出它所占有的全部资源；

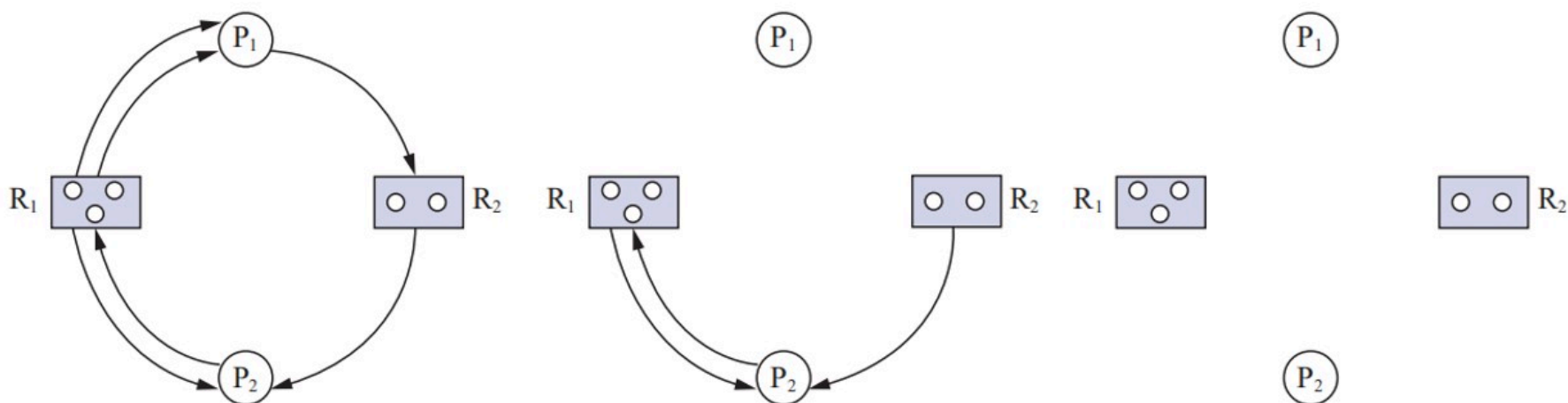
03

在进行一系列的简化后，若能消去图中所有的边，使所有进程都成为孤立结点，则称该图是可完全简化的；若不能通过任何过程使该图完全简化，则称该图是不可完全简化的。

Part 4 死锁处理方法

4.4 死锁检测

资源分配图判断死锁



Part 4 死锁处理方法

4.4 死锁检测

死锁定理



对于较复杂的资源分配图，可能有多个既未阻塞、又非孤立的进程结点，不同的简化顺序，是否会得到不同的简化图？有关文献已经证明，**所有的简化顺序，都将得到相同的不可简化图。**



S为死锁状态的充分条件是：当且仅当S状态的资源分配图是不可完全简化的。该充分条件称为**死锁定理**。

Part 4 死锁处理方法

4.4 死锁检测

数据结构类似于银行家算法（基于资源分配图简化）

Work: =Available;

$L: = \{L_i \mid \text{Allocation}_i = 0 \cap \text{Request}_i = 0\}$

for all $L_i \notin L$ do

begin

for all $\text{Request}_i \leq \text{Work}$ do

begin

Work: = Work + Allocation_i ;

$L_i \cap L$;

end

end

deadlock: = $\neg (L = \{P_1, P_2, \dots, P_n\})$;

Part 4 死锁处理方法

4.4 死锁解除

常用解除死锁的两种方法：

- 01 **抢占资源。** 从一个或多个进程中抢占足够数量的资源给死锁进程，以解除死锁状态
- 02 **终止或撤消进程。** 终止系统中一个或多个死锁进程，直到打破循环环路，使死锁状态消除为止。
 - 终止所有死锁进程（最简单方法）
 - 逐个终止进程（稍温和方法）

系统中3个进程都需要4个同类资源，一定不会发生死锁的最少资源数为（ ）。

A 9

B 10

C 11

D 12

$$M > 3 \times 2$$

B

一个作业系统中有4台打印机，每个作业需要独占2台打印机才能完成，则该系统最多可以同时执行多少作业而保证不会死锁？（ ）

A 1

B 2

☒ C 3

D 4

$$4 > N \times 1$$

关于死锁描述正确的是 ()。

- ☐ A 死锁有可能是安全状态
- ☒ B 安全状态有可能成为死锁状态
- ☐ C 不安全状态就是死锁状态
- ☒ D 死锁状态一定是不安全状态

当每类资源只有一个个体时，下列说法中不正确的是（ ）。

- ☐ A 有环必死锁 ✓
- ☐ B 死锁必有环 ✓
- ☒ C 有环不一定死锁
- ☐ D 被锁者一定全在环中

本章总结

三种调度层次★★

三种作业调度算法, 三种进程调度算法★★★★★

死锁基本概念★★★

死锁四种处理方法★★★★



课后习题

- 6.简述引起进程调度的原因
- 7.抢占式调度算法中，抢占的原则是什么？
- 12.为什么说多级反馈调度队列调度算法能较好的满足各方面用户的需求？
- 16.在批处理系统、分时系统中，各采用哪几种进程（作业）调度算法？
- 17.什么是死锁？产生死锁的原因和必要条件是什么？如何预防死锁？
- 18.在解决死锁问题的几个方法中，哪个方法最易于实现？哪个方法可使资源利用率最高？



课后习题

- (考研题) 19. 有5个进程需要调度执行，若采用非抢占式短进程优先调度算法，问这5个进程的平均带权周转时间是多少？

进程	到达时间	执行时间
P1	0.0	9
P2	0.4	4
P3	1.0	1
P4	5.5	4
P5	7	2



课后习题

- (考研题) 20. 假定要在—台处理机上执行下列作业，且假定这些作业在时刻0以1, 2, 3, 4, 5的顺序到达。请说明分别采用FCFS、RR (时间片是1), SJF及非抢占式优先级调度算法时，这些作业的执行情况 (优先级的高低顺序依次为1到5)。针对上述每种调度算法，给出平均周转时间和平均带权周转时间。

作业	执行时间	优先级
1	10	3
2	1	1
3	2	1
4	1	4
5	5	2

课程习题



课后习题

- 22. 由5个进程组成进程集合 $P=\{P_0, P_1, P_2, P_3, P_4\}$ ，且系统中有3类资源A、B、C，假设在某时刻有如下进程资源分配情况。请问当x,y,z取下列值时，系统是否处于安全状态？

(1) 1, 4, 0 (2) 0, 6, 2 (3) 1, 1, 1 (4) 0, 4, 7

进程	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	0	3	0	0	4	x y z		
P1	1	0	0	1	7	5			
P2	1	3	5	2	3	5			
P3	0	0	2	0	6	4			
P4	0	0	1	0	6	5			